Task-5:- Documentation-Deploy fe-be App on AWS + Automate Deployments.

1. Introduction

1.1 Purpose

This technical document describes the design, implementation, and operational practices of a 3-tier DevOps-based web application deployed on AWS infrastructure.

The intended audience includes:

- CTO
- Solution Architects
- DevOps Engineers
- Developers
- Resellers
- Technical Operations Teams

This document provides a complete architectural, operational, and deployment reference for replicating or scaling the solution.

1.2 Scope

This document covers:

- VPC setup and network segmentation
- EC2-based 3-tier architecture deployment
- Reverse proxy configuration using Nginx
- Backend API deployment (Node.js + Express)
- MariaDB database deployment
- Secure internal communication between tiers
- DevOps practices adopted
- Monitoring, cost optimization, and security implementation

---

2. Overview of DevOps Project

2.1 Definition

This DevOps project implements a 3-tier web application architecture using AWS infrastructure with automation, environment separation, and secure network segmentation.

Benefits include:

- Improved scalability
- Enhanced security posture
- Clear separation of concerns
- Easier monitoring and maintenance
- Infrastructure modularity

2.2 Key Principles – AWS Well-Architected Framework

The solution aligns with the 6 pillars of the AWS Well-Architected Framework:

1️⃣ Operational Excellence

- Structured deployment process
- Clear documentation
- Modular architecture
- Logging and monitoring enabled

2️⃣ Security

- Backend and DB not exposed publicly
- Private subnets for internal services
- Security group-based communication
- Reverse proxy eliminates direct backend exposure

3️⃣ Reliability

- Tier separation prevents cascading failures
- Independent instance scaling possible
- Stateless frontend

4️⃣ Performance Efficiency

- Lightweight t3.small instances
- Efficient static file serving via Nginx

- Internal private network communication

5️⃣ Cost Optimization

- Right-sized instances
- No unnecessary managed services
- EC2 overprovisioning avoided

6️⃣ Sustainability

- Minimal compute usage
- Optimized instance sizing
- Reduced unnecessary services

2.3 Goals and Objectives

- Deploy secure 3-tier architecture
- Prevent backend and DB public exposure
- Implement reverse proxy
- Enable internal-only communication
- Achieve production-style architecture

2.4 Constraints and Assumptions

Constraints

- Budget-limited implementation
- Single availability zone
- EC2-based DB instead of RDS
- No load balancer

Assumptions

- Single-region deployment (us-east-1)
- Moderate traffic expected
- Manual deployment acceptable for initial version

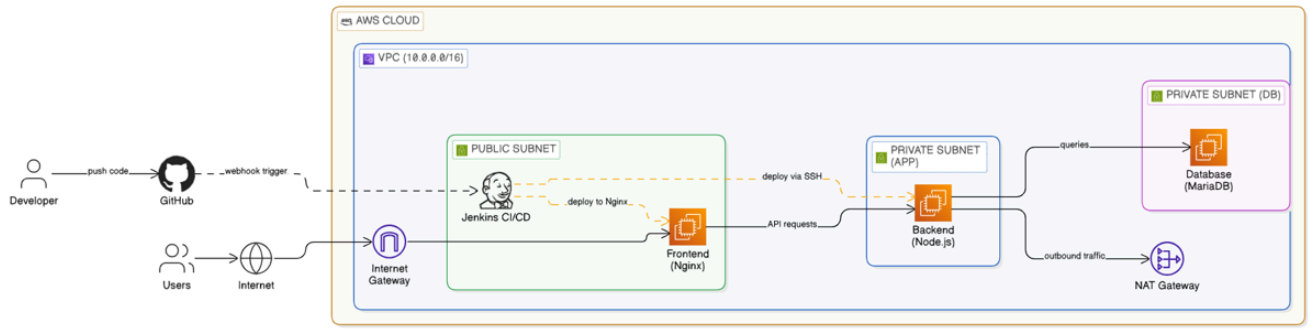
---

3. Architecture and Infrastructure

3.1 Infrastructure Overview

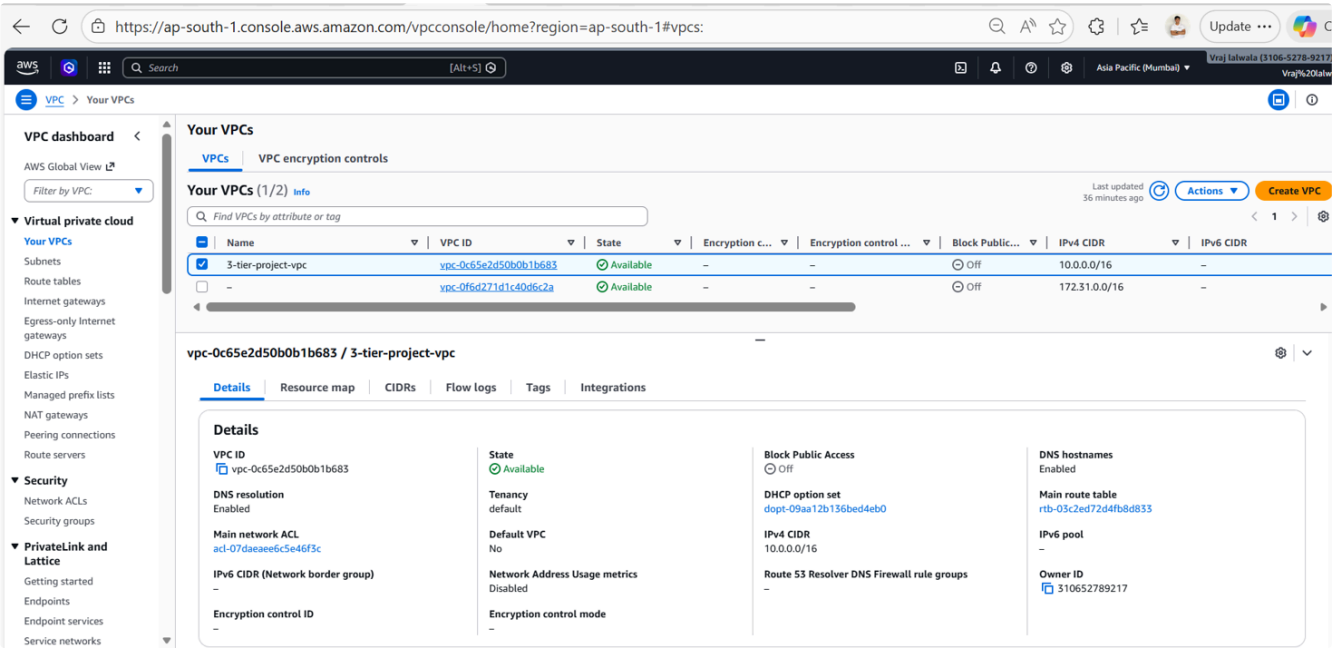
Region: us-east-1

VPC CIDR: 10.0.0.0/16



Components:

- 1 Public Subnet (Frontend)
- 2 Private Subnets (Backend & DB)
- Internet Gateway
- NAT Gateway
- 3 EC2 Instances



3.2 Environments

Environment Type: Production (Single Environment)

Future extension:

- Dev
- Staging
- Production (multi-tier)

3.3 Tools and Technologies

Layer	Technology
Frontend	React + Vite
Backend	Node.js + Express
Database	MariaDB
Web Server	Nginx
Cloud	AWS EC2
Networking	VPC, Subnets, Route Tables
Monitoring	CloudWatch
OS	Amazon Linux

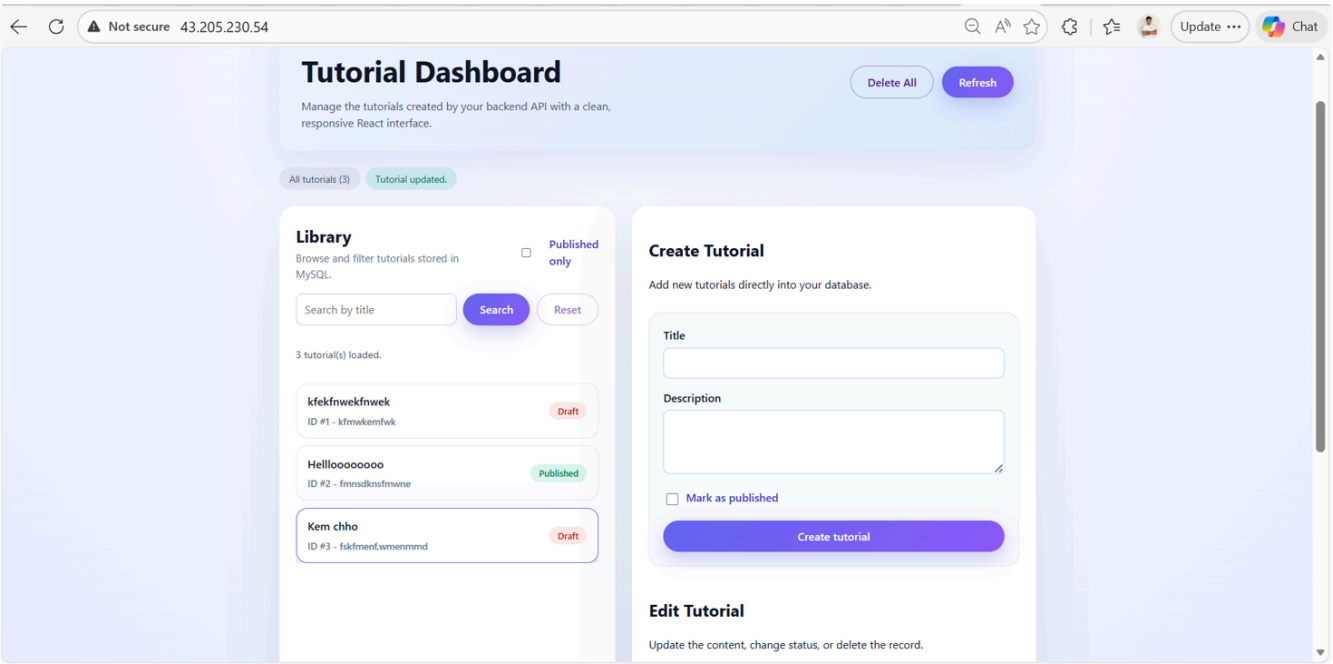
3.4 Application Architecture & User Request Flow

User Request Flow

1. User accesses Public IP
2. Nginx receives request
3. /api calls forwarded to backend private IP
4. Backend queries DB in private subnet
5. Response returned via Nginx

Flow:

User → Frontend EC2 → Backend EC2 → DB EC2



Reverse Proxy Configuration

File: `/etc/nginx/conf.d/app.conf`

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;
    location /api/ {
        proxy_pass http://10.0.135.153:8080/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
    location / {
        try_files $uri /index.html;
    }
}
```

Deployment Commands (Reusable)

Install Nginx

```
sudo dnf install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
```

Deploy Frontend

```
npm install
npm run build
```

```
sudo rm -rf /usr/share/nginx/html/*
sudo cp -r dist/* /usr/share/nginx/html/
sudo systemctl restart nginx
```

Backend Start

```
npm install
node server.js
```

Verify Reverse Proxy

```
curl http://localhost/api/tutorials
```

---

4. Development Practices

4.1 Version Control

- Git-based repository
- Centralized source control
- Branching for feature development

4.2 Continuous Integration

Current: Manual deployment
Future scope: Jenkins / GitHub Actions pipeline

4.3 Automated Testing

Basic API validation using curl and browser testing.

4.4 Code Quality

- Modular code structure
- Structured backend controllers
- Environment-based configuration

---

5. Deployment and Release Management

5.1 Deployment Strategy

Rolling update (manual).
Future improvement: Blue-Green with ALB.

5.2 Configuration Management

- Nginx config files
- Backend config files
- Environment variables
- Security group rules

5.3 GitHub Webhook

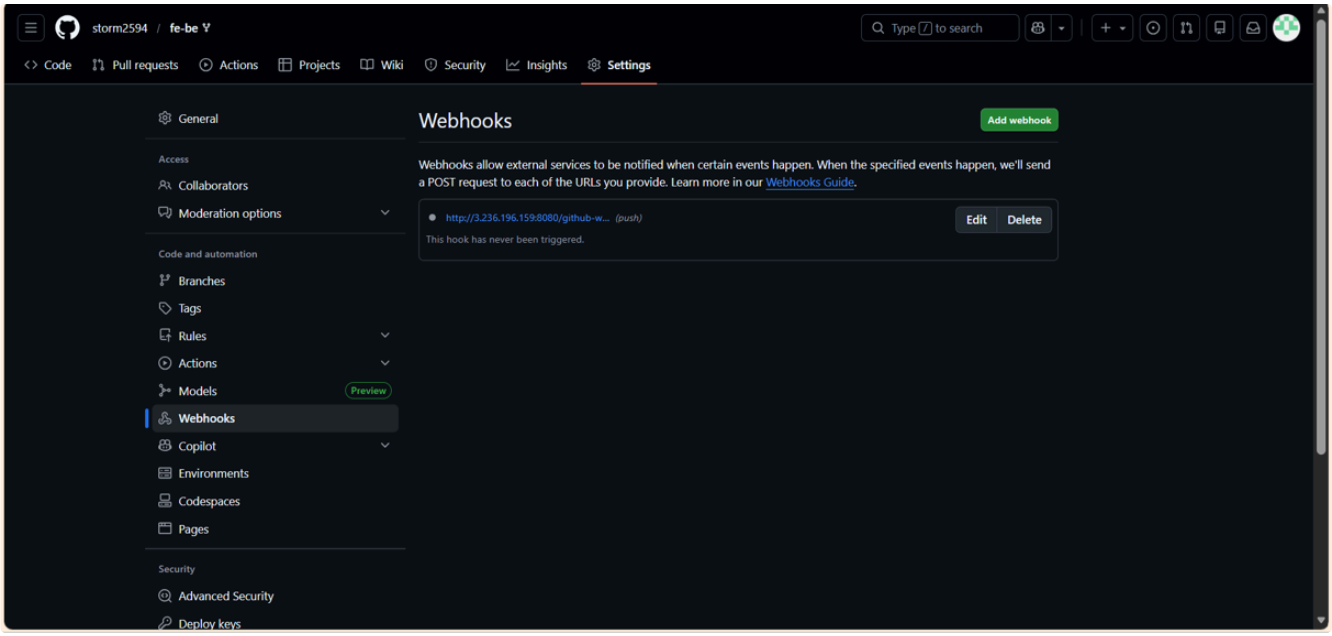
Repository:

```
https://github.com/storm2594/fe-be
```

Webhook URL:

```
http://PUBLIC_IP:9090/github-webhook/
```

Trigger: Push event

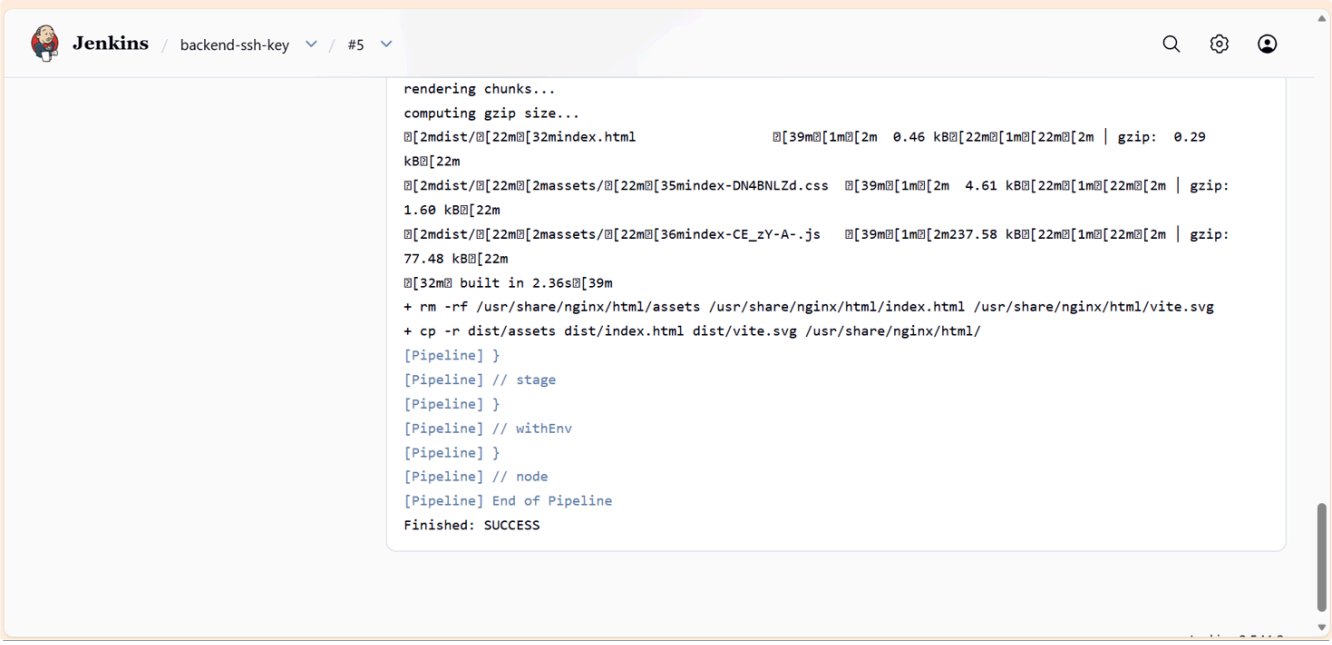


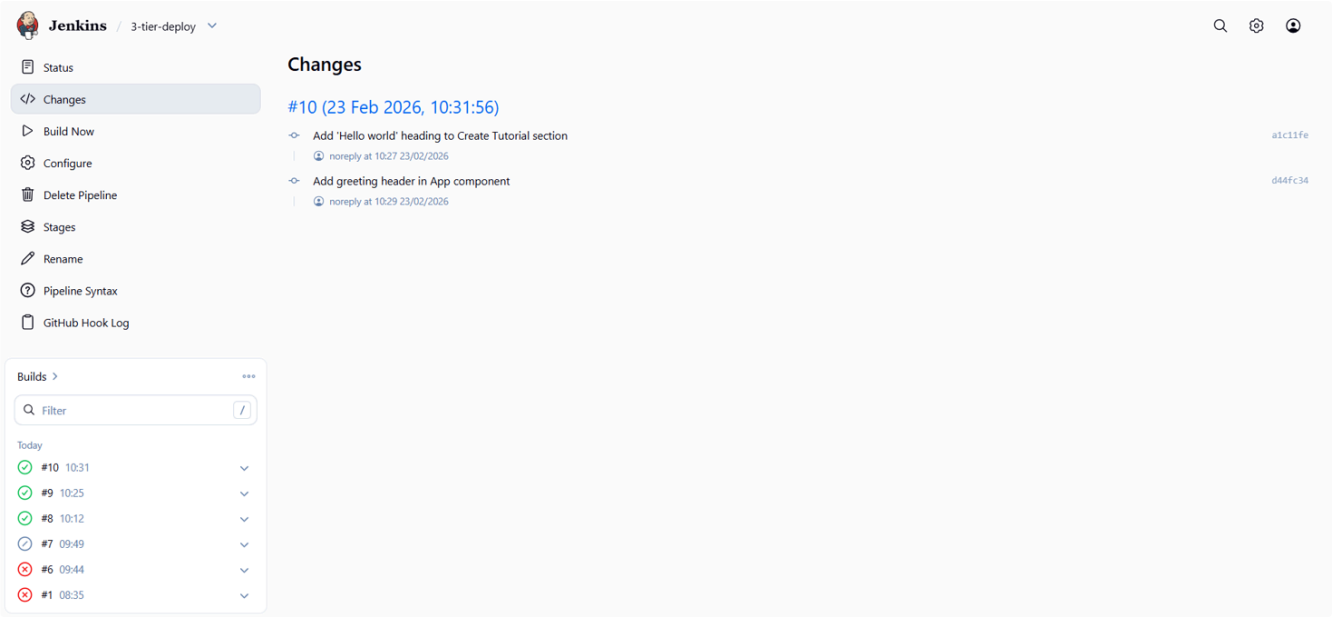
5.4 Continuous Delivery

Manual deployment in this phase.

5.5 Final Working Jenkins Pipeline

```
pipeline {
    agent any
    stages {
        stage('Pull Latest Code') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/storm2594/fe-be.git'
            }
        }
        stage('Deploy Backend') {
            steps {
                sshagent(['backend']) {
                    sh '''
                    ssh -o StrictHostKeyChecking=no ec2-user@10.0.135.153 "
                        cd ~/fe-be/nodejs-express-mysql &&
                        git pull origin main &&
                        npm install &&
                        pm2 restart backend-app || pm2 start server.js --name backend-app
                    "
                    '''
                }
            }
        }
        stage('Build & Deploy Frontend') {
            steps {
                sh '''
                cd frontend
                npm install
                npm run build
                rm -rf /usr/share/nginx/html/*
                cp -r dist/* /usr/share/nginx/html/
                '''
            }
        }
    }
```



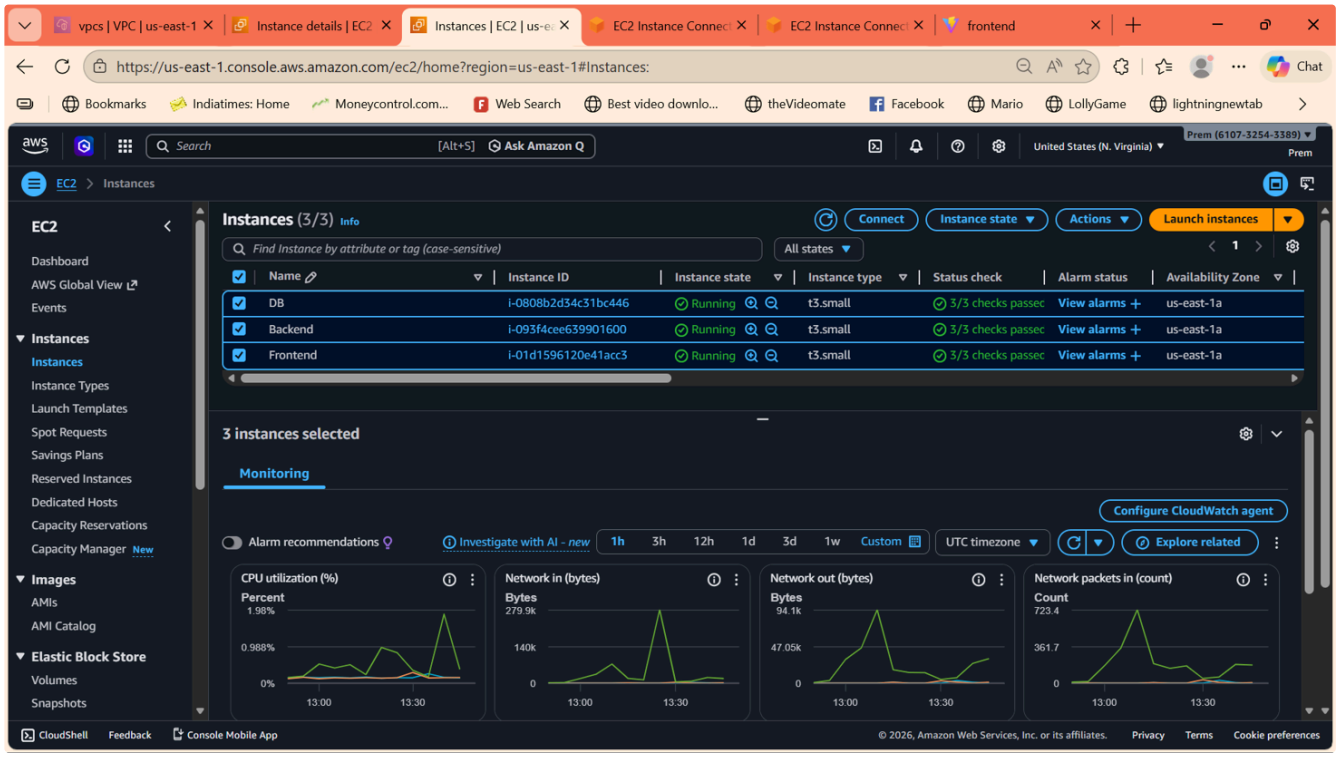


---

6. Monitoring and Incident Management

6.1 Monitoring Tools

- CloudWatch CPU
- Network metrics
- Instance health checks



6.2 Alerting

Manual review (future: CloudWatch alarms)

6.3 Incident Management

- SSH investigation
- Service restart
- Log inspection

---

7. Security & Compliance

Security Best Practices:

- Private backend subnet
- Private DB subnet
- No public DB access
- Reverse proxy hides backend
- Security group port restrictions
- IAM role-based access

---

8. Cost Optimization

Inputs Used

- t3.small instance cost
- EBS storage
- NAT Gateway cost
- Data transfer

Cost Model

3 × t3.small instances
1 NAT Gateway
1 VPC

Estimated monthly moderate usage cost: Approx $70–$120

Business Value

- Production-like secure architecture
- Scalable base for enterprise expansion

- Separation of concerns
- Low infrastructure overhead

---

10. Implementation Timeline

Phase 1 – Network Setup

- VPC
- Subnets
- Route tables
- IGW & NAT

Phase 2 – EC2 Deployment

- DB setup
- Backend setup
- Frontend setup

Phase 3 – Reverse Proxy & Testing

- Nginx configuration
- Validation
- Testing

Phase-4- CI/CD

- Jenkins

---

11. Continuous Improvement

11.1 Feedback

- Monitoring metrics review
- Performance observation
- Security review

11.2 Iterative Enhancements

Future upgrades:

- ECS deployment
- RDS migration

- ALB integration
- Auto Scaling
- CI/CD pipeline

---

12. Conclusion

12.1 Summary

This project successfully implemented a secure, scalable 3-tier DevOps architecture on AWS using EC2 instances with reverse proxy configuration.

The solution follows AWS Well-Architected best practices and ensures backend and database isolation.

12.2 Implemented

- Implemented CI/CD pipeline
- Migrate DB to RDS
- Add Auto Scaling

---

13. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____